

Barcode & QR Code Scanner

Computer Vision Project Report

Big Data & AI Engineering · 2026

Team Members

EL MANSSOURI Ismail

EL AZIZI Mohamed Taha

IBENOUAZI Ismail

EL KHAYATI Mouataze

github.com/Ismailmanssouri/barcode-qr-scanner

Python

OpenCV

pyzbar

NumPy

Tkinter

Matplotlib

1. What is this project?

This project is a **Barcode and QR Code Scanner** built with Python. It can look at any picture that contains a QR code or a barcode, find where it is in the image, read what is written inside it, and show the result on screen.

The project was built for a Computer Vision university course. Computer Vision is the field of AI that teaches computers how to **understand and analyze images**, just like a human eye does.

■ *Think of it like a phone camera that scans barcodes at a supermarket — except we built it from scratch using Python code!*

The system has three ways to use it:

- A set of **Python scripts** you can run from the terminal
- A **Jupyter Notebook** that explains every step visually
- A **desktop application** with buttons — load image, scan, save results

2. What does a QR code and barcode look like?

A **QR code** (Quick Response code) is the square black-and-white pattern you see on product packaging, restaurant menus, and posters. It stores information (like a website link) as a grid of tiny black and white squares.

A **barcode** is the series of vertical black lines you see on grocery products. It stores numbers by using lines of different widths. Common types include EAN-13 (European products), CODE-128 (shipping labels), and CODE-39 (industrial use).

Code Type	What it looks like	Where it is used
QR Code	Square grid of black/white dots	Websites, menus, tickets
EAN-13	13 vertical black lines	Supermarket products
EAN-8	8 vertical black lines	Small packages
CODE-128	Dense vertical lines	Shipping & logistics
CODE-39	Wider spaced lines	Industrial labels

3. How does the system work? (Step by Step)

The system follows a clear pipeline — a series of steps that transform a raw image into decoded results. Here is what happens at each step:

Step 1 — Load the image

We open the image file from the computer using OpenCV. The image is stored as a big table of numbers — each number represents the brightness of one pixel (0 = black, 255 = white).

```
image = cv2.imread('images/test_qr.png')
```

Step 2 — Convert to Grayscale

A color image has 3 layers (Red, Green, Blue). QR codes and barcodes are black and white, so color gives us no useful information. We convert the image to grayscale (1 layer) to make it simpler and faster to process.

```
gray = cv2.cvtColor(image, cv2.COLOR_BGR2GRAY)
```

Step 3 — Decode all codes

We pass the grayscale image to pyzbar — a specialized library that knows how to find and read QR codes and barcodes. It scans the image, finds every code, and returns its type and decoded content.

```
codes = pyzbar.decode(gray)
```

Step 4 — Get the location (polygon)

For each detected code, pyzbar also tells us exactly where it is in the image — the corner points of the code (called a polygon). This lets us draw a precise outline around it.

```
polygon = code.polygon # list of corner points
```

Step 5 — Draw bounding boxes and labels

We draw a colored polygon around each code and add a label showing the type (QR, EAN-13, etc.) and the decoded text. Each code gets a different color so they are easy to tell apart.

```
cv2.polylines(image, [pts], True, color, thickness=3)
```

Step 6 — Save the results

The annotated image is saved as a PNG file. The decoded results (type, content, position, timestamp) are also saved as a TXT file and a CSV file for easy sharing.

```
cv2.imwrite('outputs/result.png', image)
```

4. Computer Vision concepts explained simply

This section explains the image processing techniques used in the project. We describe what each technique does, why we use it, and what it would look like if you did it by hand.

4.1 Image as a grid of numbers

Every image is just a table (called a matrix or array) full of numbers. Each number is a pixel. For a color image, each pixel has 3 numbers: Blue, Green, and Red (BGR in OpenCV). For a grayscale image, each pixel has just 1 number between 0 (black) and 255 (white). Python stores this as a NumPy array.

```
image.shape = (480, 640, 3) # height × width × 3 colors
```

4.2 Grayscale conversion — why remove color?

We convert color images to grayscale because QR codes and barcodes are black and white — color adds no useful information. Removing color makes the image simpler and faster to process. OpenCV calculates grayscale by mixing the 3 color channels using a formula that matches how the human eye perceives brightness. Green contributes the most (58%) because our eyes are most sensitive to it.

```
Gray = 0.114 × Blue + 0.587 × Green + 0.299 × Red
```

4.3 Thresholding — making everything black or white

Thresholding converts a grayscale image (with 256 shades of gray) into a pure black-and-white image. Every pixel darker than a threshold becomes black, and every pixel lighter becomes white. We use Otsu's method which automatically finds the best threshold value by looking at how many dark and light pixels there are.

```
_, thresh = cv2.threshold(gray, 0, 255, cv2.THRESH_BINARY + cv2.THRESH_OTSU)
```

4.4 Edge detection — finding the boundaries

Edge detection finds the outlines of shapes in an image. The Canny algorithm works in 4 steps: (1) Blur the image to remove noise, (2) Look for places where brightness changes suddenly — those are edges, (3) Make the edges thin so they are just 1 pixel wide, (4) Remove weak edges and keep only strong clear ones. In our project pyzbar uses this idea internally to find where codes begin and end.

```
edges = cv2.Canny(blurred, threshold1=50, threshold2=150)
```

4.5 Contours — tracing the shape outline

A contour is a line that traces around the edge of a shape. Think of it like drawing around an object with a pen. OpenCV's `findContours()` finds all these outlines in a black-and-white image. For QR code detection, contours help us find rectangular regions that might contain a code.

```
contours, _ = cv2.findContours(thresh, cv2.RETR_EXTERNAL,  
cv2.CHAIN_APPROX_SIMPLE)
```

4.6 Polygon — the exact shape of the code

Once a code is detected, pyzbar gives us the exact corner points of the code as a polygon. A polygon is just a list of points connected by straight lines. Using the polygon is better than a rectangle because it follows the exact shape of the code, even if the image is tilted or distorted.

```
pts = np.array([[p.x, p.y] for p in polygon], dtype=np.int32)
```

5. System architecture

The system is divided into separate modules, each with one job. This makes the code clean, easy to understand, and easy to fix or improve.

Module	File	What it does
Installation check	check_install.py	Verifies all libraries are installed
Image loader	load_image.py	Loads and displays images with OpenCV
Core scanner	scanner.py	Detects and decodes QR codes and barcodes
Multi scanner	multi_scanner.py	Handles multiple codes in one image
GUI application	app.py	Desktop app with buttons and image preview
Report generator	generate_report.py	Creates this PDF report automatically
Jupyter notebook	notebook.ipynb	Interactive step-by-step explanation

6. The GUI application

The GUI (Graphical User Interface) is a desktop application built with Tkinter — Python's built-in library for making windows and buttons. It has a dark theme and is split into two panels: the image preview on the left and the decoded results on the right.

The application has 4 buttons:

Button	What it does
Load Image	Opens a file browser so you can choose any image from your computer
Scan Codes	Runs the full detection pipeline and shows the annotated image
Save Results	Saves the annotated image + a TXT file + a CSV file to the outputs folder
Clear	Resets everything so you can scan a new image

7. Results

The system was tested on images containing different types of codes. All supported code types were successfully detected, localized, and decoded:

Code Type	Detected?	Decoded?	Notes
QR Code	Yes	Yes	Works with URLs, plain text, contact cards
EAN-13	Yes	Yes	Standard 13-digit product barcode
EAN-8	Yes	Yes	Shorter 8-digit barcode for small products

Code Type	Detected?	Decoded?	Notes
CODE-128	Yes	Yes	Used on shipping and logistics labels
CODE-39	Yes	Yes	Industrial barcode, letters and numbers
Multiple	Yes	Yes	Multiple codes in one image, each a different color

8. Conclusion

This project shows how to build a complete image processing system step by step — from loading a raw image all the way to displaying decoded results in a desktop application.

We learned that an image is just a table of numbers, and Computer Vision is about applying mathematical operations on those numbers to extract useful information. Libraries like OpenCV and pyzbar make this much easier by providing ready-made tools that would otherwise take months to build from scratch.

The system can easily be extended in the future — for example, adding a live webcam scanner using `cv2.VideoCapture()`, or building a mobile version, or saving results to a cloud database.

■ The full source code is available at: github.com/IsmailEmanssouri/barcode-qr-scanner

9. References

- [1] OpenCV Documentation (2024) — <https://docs.opencv.org>
- [2] pyzbar Library — <https://github.com/NaturalHistoryMuseum/pyzbar>
- [3] ZBar Barcode Reader — <http://zbar.sourceforge.net>
- [4] NumPy Documentation — <https://numpy.org/doc>
- [5] Canny, J. (1986). A Computational Approach to Edge Detection. IEEE Transactions on Pattern Analysis and Machine Intelligence, 8(6), 679-698.
- [6] Otsu, N. (1979). A Threshold Selection Method from Gray-Level Histograms. IEEE Transactions on Systems, Man, and Cybernetics, 9(1), 62-66.